

YS YS

Anurag Report

 Check Plag

Document Details

Submission ID

trn:oid:::27535:140220952

Submission Date

May 24, 2026, 5:14 PM GMT+5:30

Download Date

May 24, 2026, 5:26 PM GMT+5:30

File Name

Anurag Report.pdf

File Size

1.1 MB

58 Pages

10,659 Words

56,753 Characters

*% detected as AI

AI detection includes the possibility of false positives. Although some text in this submission is likely AI generated, scores below the 20% threshold are not surfaced because they have a higher likelihood of false positives.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

SPRINT: A Remote and Timer-Controlled Smart Spraying System for Precision Agriculture

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE
OF
BACHELOR OF TECHNOLOGY
IN
ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted By:

Anurag Kumar (2K22/EC/46)

Chirag (2K22/EC/75)

Devayush Devandra (2K22/EC/88)

Under the supervision of

Dr. Yashna Sharma

(Assistant Professor)



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
DELHI TECHNOLOGICAL UNIVERSITY
(Formerly Delhi College of Engineering)
Bawana Road, Delhi-110042
May 2026

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING**DELHI TECHNOLOGICAL UNIVERSITY****(Formerly Delhi College of Engineering)****Bawana Road, Delhi-110042****CANDIDATE'S DECLARATION**

We Chirag (2K22/EC/75), Devayush Devandra (2K22/EC/88), and Anurag Kumar (2K22/EC/46) students of B. Tech. Electronics and Communication Engineering) hereby declare that the project Dissertation titled, "SPRINT(Smart Precision Remote IrrigationNetworkTechnology)" submitted to the Department of Electronics and Communication Engineering, Delhi Technological University, Delhi, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology, is original and not copied from any source without proper citation. No Degree, Diploma Associateship, Fellowship or other similar title or recognition has been awarded on the basis of this work before.

Anurag Kumar (2K22/EC/46)

Place: New Delhi

Date: 25th May,2026

Chirag (2K22/EC/75)**Devayush Devandra (2K22/EC/88)**

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING**DELHI TECHNOLOGICAL UNIVERSITY****(Formerly Delhi College of Engineering)****Bawana Road, Delhi-110042****CERTIFICATE**

This is to certify that my students Anurag Kumar (2K22/EC/46), Chirag (2K22/EC/75) & Devayush Devandra (2K22/EC/88) have performed the project work titled “SPRINT(Smart Precision Remote Irrigation Network Technology)” under my guidance & supervision and the same is a record of their project work. To the best of my knowledge, this work has not been submitted in any part or fulfillment for any Degree or Diploma to this University or elsewhere.

Dr. Yashna Sharma**(SUPERVISOR)**

Assistant Professor

Department of Electronics and Communication Engineering

Delhi Technological University

Place: New Delhi

Date: 25th May,2026

ACKNOWLEDGEMENT

This project could not have been accomplished without the help and direction of a few people.

We would like to thank the department faculty for their valuable inputs and constructive comments on the process and Prof. Neeta Pandey, HOD (ECE), Delhi Technological University for her guidance.

We are particularly grateful to our project supervisor Assistant Professor Yashna Sharma, for her guidance. Her feedback was instrumental in solving key challenges and making substantial improvements to our process.

Finally, we express our gratitude to our parents and supporters for their consistent encouragement and patience in helping us complete this project, and focus on our work.

Anurag Kumar (2K22/EC/46)

Chirag (2K22/EC/75)

Devayush Devandra (2K22/EC/88)

ABSTRACT

We have developed a Smart Spraying System to help automate the water, pesticide and liquid fertilizer distribution in farms or even small gardens. Our main goal was to cut down on manual labour by using an ESP-01 Wi-Fi microcontroller for remote operation. The hardware configuration is quite simple: ESP-01, relay, up and down voltage regulator, 12V step-up/step-down pressure pump and solenoid valves, all controlled by a central liquid source.

We didn't have to use old school timers that just spray a blind schedule, so we created a mobile app that connects to us via Wi-Fi. This enables the user to activate the spray, select the spray liquid and monitor the system in real-time. Since the system is cloud-based (Firebase), it does not require any fixed schedules and can be adjusted to fit the field's needs. The switching logic is controlled by the relay, and the voltage circuits ensure the stability of power supply to the various components. We created a modular design so we can add soil moisture sensors and weather APIs later and prevent it from spraying during rain. Basically, this is good for any size garden from a terrace garden to an open field. It is an affordable and easily scalable solution to enter the precision farming and prevent wasting resources.

List of Figures

Figure 1.1 : Comparison of manual vs. smart spraying practices	3
Figure 2.1 : Diagram showing smart spray farming	5
Figure 2.2 : Diagram showing Firebase – ESP8266 architecture	6
Figure 2.3 : Diagram depicting ESP8266 information exchange with Firebase	7
Figure 2.4 : WiFiManager user interface	8
Figure 2.5 : AgriSpray+ Smart Spraying System – full circuit and setup	14
Figure 3.1 : ESP-01 with pinout diagram	16
Figure 3.2 : ESP-01 relay module	16
Figure 3.3 : Solenoid valve	17
Figure 3.4 : Pressure pump	17
Figure 3.5 : 12 V power adapter	18
Figure 3.6 : MT3608 voltage booster module	18
Figure 3.7 : LM2596 voltage step-down module	19
Figure 3.8 : ESP8266 – Firebase RTDB architecture	20
Figure 3.9 : System backend architecture	21
Figure 4.1 : Control logic (pseudocode)	26
Figure 5.1 : ESP-32 development board	30
Figure 5.2 : AiThinker ESP32-CAM module	30
Figure 5.3 : Raspberry Pi 3B+	31
Figure 5.4 : NodeMCU development board	32
Figure 7.1 : ESP-01 firmware initialisation	42
Figure 7.2 : Controller card layout	42
Figure 7.3 : Android app user interface	43
Figure 7.4 : Internal circuit diagram	43

LIST OF ABBREVIATIONS AND SYMBOLS

Symbol	Definition
ESP	Espressif Systems Protocol
GPIO	General Purpose Input/Output
UI	User Interface
API	Application Programming Interface
DC	Direct Current
AC	Alternating Current
IoT	Internet of Things
JSON	JavaScript Object Notation
RTDB	Realtime Database
SSID	Service Set Identifier
EEPROM	Electrically Erasable Programmable Read-Only Memory
OTA	Over-the-Air (firmware update)
IDE	Integrated Development Environment
LED	Light Emitting Diode
MCU	Microcontroller Unit

List of Tables

Table 5.1 : The Results Summary of Findings	36
Table 7.1 : Commercial Comparison of HiOazo Smart Controller and SPRINT	41
Table 7.2 : Components used in the final prototype	44

Contents

Candidate's Declaration	i
Certificate	ii
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
List of Figures	v
List of Abbreviations and Symbols	vi
List of Tables	vii
CHAPTER 1: Detailed Problem Statement	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Problem Statement	3
1.4 Thesis Overview	4
CHAPTER 2: Literature Survey	5
2.1 Introduction	5
2.2 Related Work and Technology	6
2.3 Libraries and Tools Used	6
2.4 Advantages of Using Open-Source Platforms	10
2.4.1 Cost Efficiency	10
2.4.2 Community Support	11
2.4.3 Rapid Prototyping	11
2.4.4 Modularity and Customisation	11
2.4.5 Educational Value	11
2.4.6 Scalability	11

2.4.7	Use of IoT in agriculture for smart farming.	11
2.4.8	Remote Monitoring and Control	12
2.4.9	Data-Driven Decision Making	12
2.4.10	Automation of Agricultural Tasks	12
2.4.11	Resource Optimisation	12
2.4.12	Scalability and Integration	12
2.4.13	Internet Connectivity through Wi-Fi in Rural Areas	13
2.4.14	Insufficient Memory Capacity on ESP-01	13
2.4.15	Unstable Voltage in Power Supply	13
2.4.16	Hardware Integration	13
2.4.17	Cloud Dependency	13
2.4.18	Security Issues	14

CHAPTER 3: Requirements Specification **15**

3.1	Introduction	15
3.2	Hardware Requirements	15
3.2.1	Microcontroller Unit	15
3.2.2	Actuation Components	16
3.2.3	Power Supply and Regulation	17
3.2.4	Additional Components	19
3.3	Software Requirements	19
3.3.1	Development Tools	19
3.3.2	Communication Libraries	20
3.3.3	Data Handling	20
3.3.4	Backend & Mobile Interface	20
3.3.5	Debugging Tools	21
3.4	Functional Architecture	21
3.4.1	Application Layer (User Interface)	22
3.4.2	Cloud layer (Firebase)	22
3.4.3	Control layer (ESP-01)	22
3.4.4	Actuation layer (Relays, pumps, solenoid valves)	22

CHAPTER 4: Model Proposed **23**

4.1	Introduction	23
4.2	System Overview	23
4.3	Block Diagram	25
4.4	Working Principle	25
4.4.1	User Command Initiation	25
4.4.2	Firestore Modification	25
4.4.3	Data Polling by the ESP-01	25
4.4.4	Logical Decision Making	25
4.4.5	Activation of Relay	26
4.4.6	System Shutdown	26
4.5	Control Logic and Flowchart	26
4.6	Benefits of Proposed Model	27
4.6.1	Low-cost Hardware Design	27
4.6.2	Reduced Plumbing Complexity	27
4.6.3	Wireless Operation	27
4.6.4	Scalability	27
4.6.5	Power Management	27
4.6.6	Intuitive User Interface	27
4.6.7	Safe Hardware Usage	28

CHAPTER 5: Experimentation and Results **29**

5.1	Microcontroller Assessment	29
5.1.1	ESP-32	29
5.1.2	AiThinker ESP32-CAM	30
5.1.3	Raspberry Pi 3B+	30
5.1.4	NodeMCU	31
5.1.5	H5 Stack	32
5.1.6	Final Choice: ESP-01	32
5.2	Power Supply Testing	33
5.2.1	AC to DC Adapters	33
5.2.2	Voltage Regulation	33
5.2.3	Final Power Configuration	33
5.3	Relay Trials	34
5.3.1	Relay Module Testing	34

5.3.2	Pressure Pump Tests	35
5.4	Connectivity and Cloud Communication Test of ESP-01	35
5.5	Cycle Tests For The Spraying System	35
5.6	Environmental and Field Adaptability Tests	36
CHAPTER 6:	Conclusion	38
6.1	Pros and Cons	38
6.2	Future Scope	39
6.3	Final Remarks	40
CHAPTER 7:	COMMERCIAL COMPARISON	41
Appendix		42
A.1	Source Code Snippet	42
A.2	Mobile App Screenshot	42
A.3	System Circuit Diagram	43
A.4	Components Used	44
References		44

CHAPTER 1

Detailed Problem Statement

1.1 Introduction

The agriculture sector is still the main pillar of the Indian economy today. This accounts for a colossal share of GDP and the lives of majority of our population. But on the ground, the methods are still limited to the past for decades. Even the small farms rely nearly exclusively on manual labour for basic operations such as chemical spraying, irrigation etc. There are a number of serious problems with this old habit. Waters are wasted regularly, coverage of chemicals is very inconsistent and a lot of time is wasted every day. Not only is everyone else at risk of health problems, the workers are at constant danger of health problems from the toxic pesticides themselves.

Manual spraying has the greatest disadvantage in terms of inefficiency. It is virtually impossible to ensure even application when a chemical has to be applied by a walk-behind heavy tank in poor fields. At times it happens that plants are completely covered with chemicals while the surrounding plants remain dry. Long term effects of this erratic coverage include reduced yields and poor soil quality. The chemicals are also destroyed by evaporation as spraying is often carried out during the peak daylight hours. Overnight downpours may simply get rid of the high cost treatments altogether. The same issue with rooftop farming and terrace gardens exists in urban areas. The smaller-scale systems are in dire need of automated and compact tools since conventional commercial farming equipment is too cumbersome for a home environment.

Our project addresses this very technological gap. A compact microcontroller was used to successfully develop a Smart Spraying System. Easy-to-acquire hardware and standard consumer IoT capabilities made it possible to automate the delivery of water and chemicals. With affordable hardware and standard consumer IoT capabilities, the automation of water and chemical delivery was possible. The ESP-01 is the controlling component of the entire system. The system is directly connected to a Firebase Realtime Database for real-time data synchronization. Moreover, we created a special Application for Android, so that the user can activate the spray mechanism remotely from any smartphone connected to the internet. The actual physical execution uses a 12V DC pressure pump, a 24V DC solenoid valve for the flow control and a standard relay module for the electronic switching. The stepping down converter, LM2596, functions together with the MT3608 stepping up converter for

smooth voltage management. These particular components were selected due to their high availability in their local markets and low cost. Its final solution is a reliable and efficient system with minimal human oversight of its operation to ensure safety.

The software backend uses very light digital tools such as the Mobizt Firebase ESP Client library. WiFiManager simplifies the process of setting up a network so you don't have to deal with a common headache. The system automatically starts a captive portal to quickly set up the network credentials when needed, without manual flash. From the beginning, we maintained a completely modular architecture. This current build only toggles one relay, but there is very easy expansion. Future soil moisture sensors and live weather data streams can be easily connected to this framework, which the machine can use to determine when to spray on its own. The main goal is to eliminate guesswork and automate precisely, regardless if the system is in a commercial polyhouse or a backyard garden.

1.2 Motivation

This project began with a rather mundane observation: on most of the small farms and in most of the terrace gardens, the spraying is either manual or controlled by a simple timer. Both of these are unhealthy for different reasons. Manual spraying is tiring, imprecise and exposes the applicator to pesticide for long periods of time. Timer based systems are a bit better, but still very stupid — they will spray whether it was an hour ago or the tank is empty.

We spoke to a few who are directly involved in this. One of them operates a rooftop facility in West Delhi, and was using a mechanical timer for the drip. His principal objection was that he did not know whether it had caused while he was at work. Another farmer, a small farmer from Haryana, said that it was a common issue of irregular spraying, some rows getting soaked and others minimally covered. They didn't have any problem with technology, it was just that they couldn't find anything simple and cheap enough.

This is what we had in mind. The ESP-01 was cheap enough (less than Rs.150) and Firebase is a solution that doesn't require us to create our own server for the remote control mechanism. It was then connected by the Android app. The intention was not to create anything complicated — just something that's actually used.

Aspect	Manual Spraying	Smart Spraying
Cost	Lower initial cost; requires simple handheld tools.	Higher initial cost due to automation and IoT devices.
Ease of Use	Simple to operate; no technical knowledge required.	Requires basic technical setup but offers automation.
Precision	Application may vary based on human judgment.	Delivers exact amounts, reducing wastage.
Labor Effort	Physically demanding; requires manual handling.	Reduces human labor through automation.
Control	Farmer directly applies chemicals.	Remote operation via mobile apps or automated systems.
Coverage	May be uneven due to human error.	Ensures uniform application across crops.
Environmental Impact	May lead to excess chemical use and runoff.	Optimized application reduces environmental harm.

Figure 1.1: Comparison of manual vs. smart spraying practices

1.3 Problem Statement

The main aim of this project is to design and develop a smart Microcontroller based spraying system. A custom Android application that communicates to Firebase via Wi-Fi enables remote watering, fertilizing and pesticide applications. The biggest advantage is the instant, real-time control, and this setup would get rid of the stiff, hardware timers forever. Additionally, the entire structure is extremely flexible as it can be easily integrated with environmental sensors and weather-related automation systems in the future.

More concretely, the prototype must satisfy the following requirements:

- Input to the system: the Firebase Realtime Database should receive commands from a user-operated Android application.
- Controller: the ESP-01 should fetch these commands in near real time and act as the primary controller for activating the output devices.
- Output from the system: based on the values read from Firebase, the relay module should switch the solenoid valve and the pressure pump on and off, allowing controlled spraying

of fluids. Beyond functionality, the system should also be power-efficient, easy to install, and compatible with mobile networks via local Wi-Fi. These traits together make it equally suitable for rural and urban agricultural setups.

1.4 Thesis Overview

This thesis is organised into seven chapters. Each one builds on the previous and walks the reader through both the understanding behind the system and the engineering choices that went into it.

Chapter 1: Introduction. Clearly states the problem space, background, motivation and the scope and goals of the project.

Chapter 2: Literature Review. Summarizes the state of the art, identifies current limitations and surveys the existing work on agricultural automation using particularly IoT and microcontroller-based systems.

Chapter 3: Software Requirements Specification. Records detailed requirements for hardware and software, functional and non-functional requirements for system, and user interface and control logic.

Chapter 4: System Design and Implementation. Describes the architecture of the system, such as a block diagram, a circuit design and communication protocols. It also explains the reasons for choosing the components like the ESP-01, relay module, voltage converter and the solenoid valves.

Chapter 5: Testing and Results. Provides analysis of the prototype's performance. It includes test cases, troubleshooting, and validating control in the mobile app and Firebase.

Chapter 6 Conclusion and scope of work. Summarises project outcomes and limitations and suggests improvements like sensor integration, weather dependent automation and offline control modes, and solar energy solutions.

Chapter 7 – References and Appendix. Includes an overview of all the academic, technical and open-source resources utilized during the project, as well as additional resources (circuit diagrams, code snippets, app screenshots, etc.).

CHAPTER 2

Literature Survey

2.1 Introduction

Agriculture in India has a vast expanse, but the instruments in use for most small farms have not undergone any drastic change for decades now. In terms of basic operations such as irrigation and pesticide application, manual methods are the only methods available which cause some consistent problems. There is wastage of water, improper application of chemicals as well as direct contact with harmful chemicals by workers for several hours.

The question is not one of energy alone. A worker moving around the rough terrain with a pesticide container cannot possibly cover all areas equally. Some places will receive too much and others not enough. Improper timing while using pesticides in mid-day when it is convenient causes evaporation of a considerable quantity of the mixture. If rain falls in a few hours, the whole process has to start from scratch. The same problems can be seen in urban settings of small roof gardens, since large agricultural machinery is out of the question.

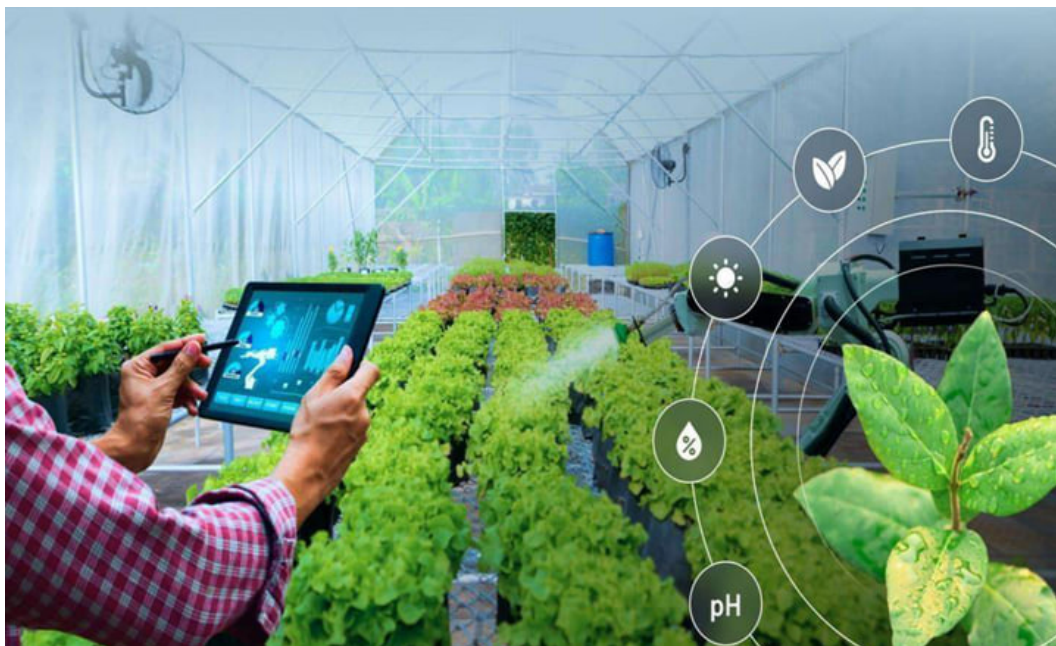


Figure 2.1: Diagram showing smart spray farming

2.2 Related Work and Technology

Many research studies and actual projects have illustrated the benefits of implementing the concept of smart irrigation using microcontrollers. Some of the consistent themes in this regard include:

- 1) The use of NodeMCU and ESP8266 modules for controlling actuators via smartphone apps;
- 2) Firebase Realtime Database usage for monitoring and remote controlling;
- 3) Solenoid valves and relay modules usage for automating the process of fluid delivery; and
- 4) Mobile graphical interfaces usage created with MIT App Inventor, Android Studio or even a custom solution.

The guide referred above [TheRandomNerdTutorials](#) illustrates real-time bi-directional communication between Firebase and ESP module. Among other things we have taken away from it included:

- 1) How to use Firebase along with the right setup of its authentication and data structure;
- 2) How to communicate with the Firebase using read and write operations from the ESP-01; and
- 3) How to establish reliable Wi-Fi connection.

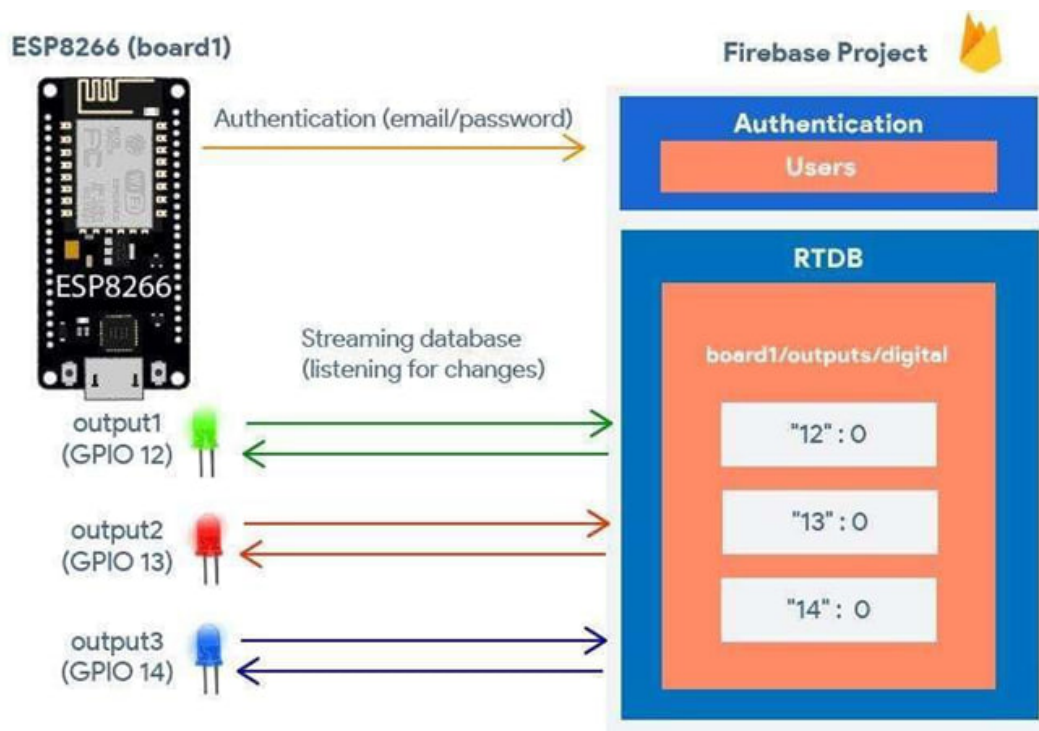


Figure 2.2: Diagram showing Firebase – ESP8266 architecture

2.3 Libraries and Tools Used

A cloud-controlled smart spraying system was successfully implemented on a resource-limited module, such as the ESP-01, relying carefully on the use of software libraries. Each and every tool was required

to be the best fit as the flash memory and RAM on this particular equipment is extremely low. We explicitly selected libraries that are small, well documented, and proven to be stable in production environments. A list of the tools and software libraries used throughout this project are explained below.

1. Firebase ESP Client by Mobizt (<Firebase_ESP_Client.h>)

This is the backbone of all communication between the ESP8266/ESP-01 module and the Firebase Realtime Database. The small ESP module can execute full CRUD (Create, Read, Update, and Delete) operations directly on Firebase data nodes — this is supported in full by this software, which is maintained by Mobizt. Secure, token-based authentication and native support for REST and streaming ensures safe data transmission. The following main prompted our choice of this particular library:

- Real-Time data synchronisation – all changes done in the mobile app are immediately synced to the device.
- Efficient memory usage: light enough to co-exist with WiFiManager and rest of firmware on ESP01 we used.
- Inbuilt reconnection and error handling - the client will automatically reconnect if it fails, especially important for unattended deployments in the agricultural setting.



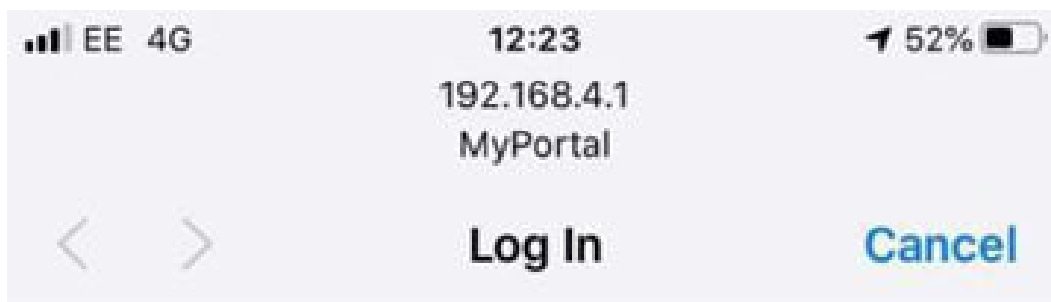
Figure 2.3: Diagram depicting ESP8266 information exchange with Firebase

2. WiFiManager by tzapu (<WiFiManager.h>)

One obvious drawback in this regard is the situation when you have to embed the Wi-Fi credentials right into the device firmware. Such an approach is absolutely unrealistic when it comes to the practical

implementation of an agricultural instrument, especially taking into account how easy it might be for a person to change his/her router settings, passwords, or just move to a different location altogether. The WiFiManager library makes such problems a thing of the past because of its capability to open the captive portal at the time of the initial booting of the device. With the help of this portal, the user will be able to set up his/her network SSID and password using a web browser interface.

- Credential storage: SSID and its password are stored in EEPROM for further use.
- Reset and recovery options: Wi-Fi can be reconfigured easily using GPIO signal.



MyPortal

WiFiManager



Figure 2.4: WiFiManager user interface

3.ESP8266WiFi (<ESP8266WiFi.h>)

This is a basic Wi-Fi library that comes with the ESP8266 Arduino, which takes care of all basic networking operations on our device. It handles the basics under the hood – whether to successfully connect to a local access point, obtain a new IP address when needed, and monitor the overall connection stability. This is the only library that is crucial for our work and has three functions:

- SSID scanning, from which the captive portal could display the list of available networks;
- signal monitoring, aiding in debugging poor-connectivity situations; and

The firmware should also be able to respond to disconnections or IP conflicts, so it's important to have event handling capabilities.

4.ArduinoJson (<ArduinoJson.h>)

All the entries in the Firebase database are stored in a native JSON (JavaScript Object Notation) format. ArduinoJson provides efficient encoding and decoding of these data structures exactly what we need on such a tight-fitting ESP module.

The major advantages of making use of this library of this library include:

- memory efficiency (small heap on the ESP-01); and
- ease of use, clean set of APIs for serialising and deserialising JSON data; and
- smooth interoperability with the Firebase client library.

5.ESP8266WebServer (<ESP8266WebServer.h>)

It is possible to create a small HTTP server on the ESP-01 using this library. It was useful to check if the firmware was alive even if Firebase was temporarily unavailable, as we used it primarily for debugging purposes to create a local status page, accessible over the LAN. It has useful applications such as:

Offline access without Firebase – for offline testing and diagnostics; and

Displaying simple HTML/JSON content to respond to a simple control command or status information.

6.<DNSServer.h>

In conjunction with WiFiManager, it also hijacks DNS requests and steers them towards the captive portal. It ensures:

A seamless user experience as the configuration page opens automatically on any browser; and a network-agnostic configuration, so the user is not required to know the device's ip.

7.<ESP_EEPROM.h>

Persistent data (Wi-Fi credentials, calibration values or Device IDs) is stored in EEPROM (Electrically Erasable Programmable Read-Only Memory). Retaining data across reboots or power failures: key applications include data persistence settings.

Custom configuration storage, user-defined spraying preferences can be saved.

8.<NTPClient.h> and <WiFiUdp.h>

To get the current time from an NTP server over the internet, these libraries are used, which is very handy for time-stamped logging or scheduling spraying tasks. They are commonly used for: • emulation of real-time clock in real-time; and

- scheduling events for time based automation of spraying.

9.<ESP8266HTTPClient.h>

This library provides the capability of making HTTP calls to external servers. It can be used to log data to third party platforms or interact with external APIs. Typical uses include:

- data logging to cloud services; and
- Downloading configuration files or firmware from remote locations.

There are several benefits to choosing open-source platforms: The use of open-source platforms offers several advantages:

2.4 Advantages of Using Open-Source Platforms

Prototyping and hardware development have been changed almost beyond recognition by open-source platforms. These common assets reduce the entry hurdle for technically entering the field and visibly accelerate the innovation rate. As part of this project we have used a number of open sourced libraries and tools, which has brought several practical and technical benefits directly to us.

2.4.1 Cost Efficiency

The financial viability is obtained mainly due to the use of free major libraries and development tools in this project. With this open access, there is no requirement for any proprietary software or hardware-specific SDKs, and it is free. This means that the whole system can be built, tested and deployed within a very limited student budget.

2.4.2 Community Support

A large group of developers around the globe are keeping the project's libraries. As we have several helpers on GitHub, we never have unsolvable errors.

2.4.3 Rapid Prototyping

Prototyping is very easy and accessible in platforms such as Arduino IDE. Simply connect the module to the laptop and it is instantly recognized.

2.4.4 Modularity and Customisation

The open source libraries that we employed are extremely modular. They give no issues when chained together with the other libraries. There are many libraries readily available on GitHub to use and develop.

2.4.5 Educational Value

It gets a much deeper understanding of networking protocols, memory management, and embedded programming through studying the open-source code of these libraries. It is a hands-on analysis and provides a huge technical benefit in learning to manage real-world hardware limitations.

2.4.6 Scalability

Open source solutions are always scalable. This firmware is coded in the same way as for the ESP-01, but can be adapted for more capable boards such as ESP32 with virtually no modification.

2.4.7 Use of IoT in agriculture for smart farming.2.5.1 Agriculture use of IoT for smart farming.

Replace, monitor and control agricultural systems with remote connectivity, is the primary change that IoT has brought to the agricultural system. This transformation was all we were looking for in a project such as ours. Without network connection the ESP-01 has little to no value whatsoever, but once connected to the network, it is a controller that you can control from anywhere.

2.4.8 Remote Monitoring and Control

The biggest benefit we had was the fact that we didn't have to be physically there to run the system. After the Firebase connection became stable, triggering a spray cycle from a cell phone seemed to be the same from standing next to the hardware or across campus.

2.4.9 Data-Driven Decision Making

Integration with cloud platforms makes it possible to store and analyse historic spraying data. These are log records which with time can assist the developer in determining the optimum spraying time for certain crops. It also helps to identify system anomalies or anticipate a critical hardware failure much easier in advance.

2.4.10 Automation of Agricultural Tasks

Highly inconsistent coverage of the crop is also achieved due to the timeconsuming work of traditional sprayer, where the operator had to do spraying work continuously. The Internet of Things (IoT) makes it possible to implement fully automated operating modes that rely on exact schedules, local weather information or on-the-fly sensor data. Thanks to this smart automation, the application of chemicals and water is exactly even, reliable and timely.

2.4.11 Resource Optimisation

Only active pumping uses power and liquid in the system. The ESP-01 uses very little power in idle mode and the pump and valve are shut down in idle mode. Cloud connected control as opposed to a timer-driven system, which will water on a set schedule whether soil is dry or not, will only use resources when a command is sent. If a soil moisture sensor was added a future version of it would be further as that decision would be automatic.

2.4.12 Scalability and Integration

Our design allows for easy expansion without too many changes. Adding another relay channel would allow a second solenoid to be controlled independently. The soil moisture sensor, DHT22 (temperature and humidity sensor) or rain sensor can all be connected to the same Firebase database and have an impact on the control logic. The biggest hardware constraint is of the ESP-01, as it only has 2 GPIO pins available, so an upgrade to an ESP-32 would be the obvious next step if more than one or two outputs need to be controlled concurrently.

Accessibility and Inclusiveness

The total component cost not only of this build remained less than Rs.1500 but was also under budget. The cost of the ESP-01 itself is in the range of Rs.80-100 in Lajpat Rai. When a system of this magnitude produces the data, it's free under the Spark plan on Firebase. Android Studio comes with no cost. This is important, because a lot of the products currently available in the market for agriculture are aimed at commercial farms and not for those who have a small plot or a terrace garden and want to automate it without spending thousands of rupees on a proprietary Challenges in Implementation.

The project demonstrated the potential for cloud-based control and IoT to be used reliably, but encountered some practical issues.

2.4.13 Internet Connectivity through Wi-Fi in Rural Areas

Continuous connectivity with internet is essential for ESP-01 in order to be able to send information to Firebase. Large time intervals for executing commands, or no response at all from the system occurs, since there isn't always a good Wi-Fi connection in the rural farmlands due to constant loss of signal.

2.4.14 Insufficient Memory Capacity on ESP-01

There is only 512 KB capacity for flash memory storage on ESP01. It is too less to accommodate any additional coding apart from the actual code itself.

2.4.15 Unstable Voltage in Power Supply

The ESP module is very sensitive when it comes to voltage supply and therefore can lead to malfunctioning of the system, especially in rural areas where the product will be used.

2.4.16 Hardware Integration

Careful integration with respect to current and voltage must be achieved for control of solenoid valves and pumps.

2.4.17 Cloud Dependency

The system that we designed needs to be online 24x7 to function. The project area should have WiFi connection for the system to connect with Firebase in the cloud without any problem.

2.4.18 Security Issues

There are major concerns of intercepting any data sent through Wi-Fi networks unless it is secured. Token based authentication provided in Firebase provides reasonable security but does not offer total protection. Implementation of HTTPS and Over-The-Air updates in the future will strengthen the overall security system against any threats.

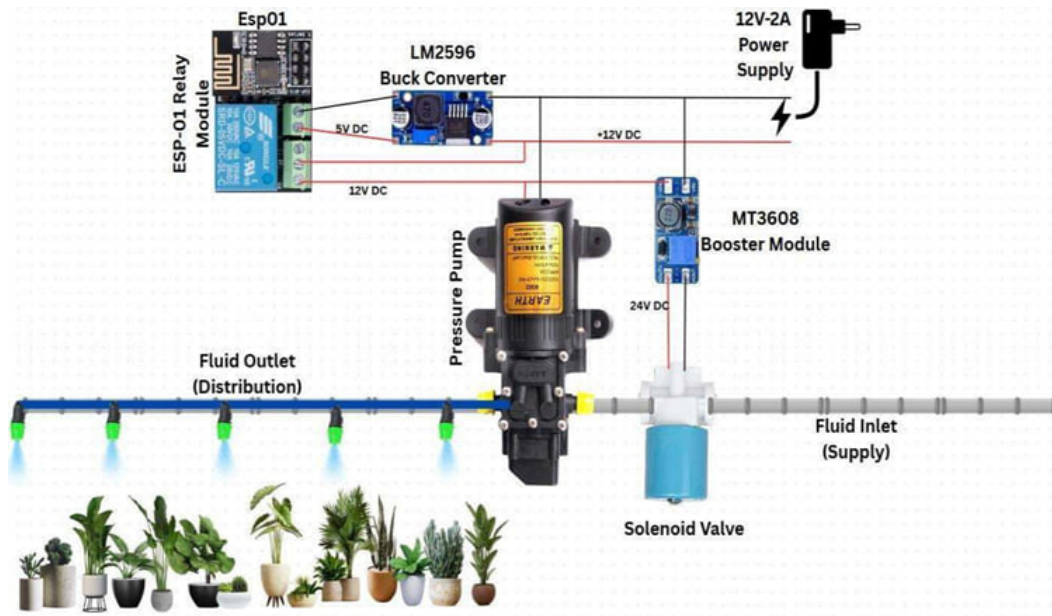


Figure 2.5: AgriSpray+ Smart Spraying System – full circuit and setup

CHAPTER 3

Requirements Specification

3.1 Introduction

All final system requirements should fulfill the following requirements at the same time: reliability, good scalability capabilities, and cost effectiveness. This is very important because the final instrument should be easy to use by common users, including farmers and people from villages, who most likely do not have any engineering knowledge.

All the physical instruments, electronics modules, and libraries employed in this project will be described in this chapter. The connection with the Firebase Realtime Database and driving all peripheral devices, such as pumps and valves, will be performed with the help of the central ESP-01 microcontroller. The choice of power parts was done carefully to be able to handle all required voltages correctly; however, it is the software part that allows controlling the irrigation system remotely using a mobile device.

3.2 Hardware Requirements

3.2.1 Microcontroller Unit

ESP-01 (ESP8266): The entire task of central processing for this device is done through this microcontroller itself. Through this inexpensive yet extremely small module, wireless communication capabilities can be achieved in conjunction with the cloud-based platforms like Firebase. This module requires a steady power supply of 3.3V to operate. It supports the usage of two GPIOs, which are named GPIO0 and GPIO2, and we used them in combination with our mechanical relay module and LEDs.

Instantaneous actuation takes place as the ESP-01 is always listening to the Firebase Realtime Database for any changes in states being sent. This module has a robust connection established underneath to ensure that there is instantaneous actuation upon pressing of the button by the user. It is the perfect choice for IoT-based applications due to its reliability and low cost of implementation.

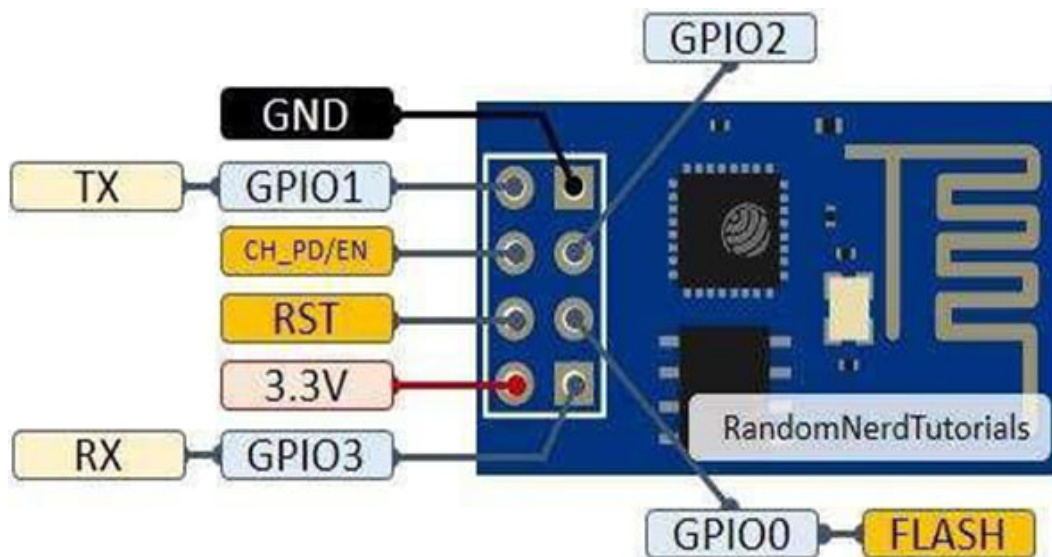


Figure 3.1: ESP-01 with pinout diagram

3.2.2 Actuation Components

The actuators carry out the mechanical instructions received from ESP-01 that entail the switching of the sprayer pump and valve solenoids.

- **Relay Module (5V, 1 channel):** The relay module does the job of handling the difficult task of interfacing the ESP-01 with high-voltage circuits such as 12V DC solenoid valves and pumps. The sensitive electronics are safe since the relay module maintains isolation between the microcontroller circuitry and the current drawing hardware components. It uses direct power from ESP-01 and gets triggered with each GPIO command.



Figure 3.2: ESP-01 relay module

- **Solenoid Valves (12V DC):** The solenoid valves control the movement of liquids such as water, insecticide, and fertilizer that are stored separately in their individual tanks. This is done by activating or deactivating the solenoids according to signals received from Firebase via the relay.



Figure 3.3: Solenoid valve

- **Pressure Pump (24V DC):** This element provides sufficient pressure to enable liquid spraying on crop or designated areas. It can be turned on or off using the relay module, and it plays a crucial role in providing sufficient coverage during spraying.



Figure 3.4: Pressure pump

3.2.3 Power Supply and Regulation

- **AC to 12 V DC Power Adapter:** No battery is used for powering this system. Instead, a power adapter provides power for the pump, valves, and voltage regulation units.



Figure 3.5: 12 V power adapter

- MT3608 Boost Converter: The MT3608 boost converter boosts the power rail voltage from 5V to 24V DC to run the pressure pump.

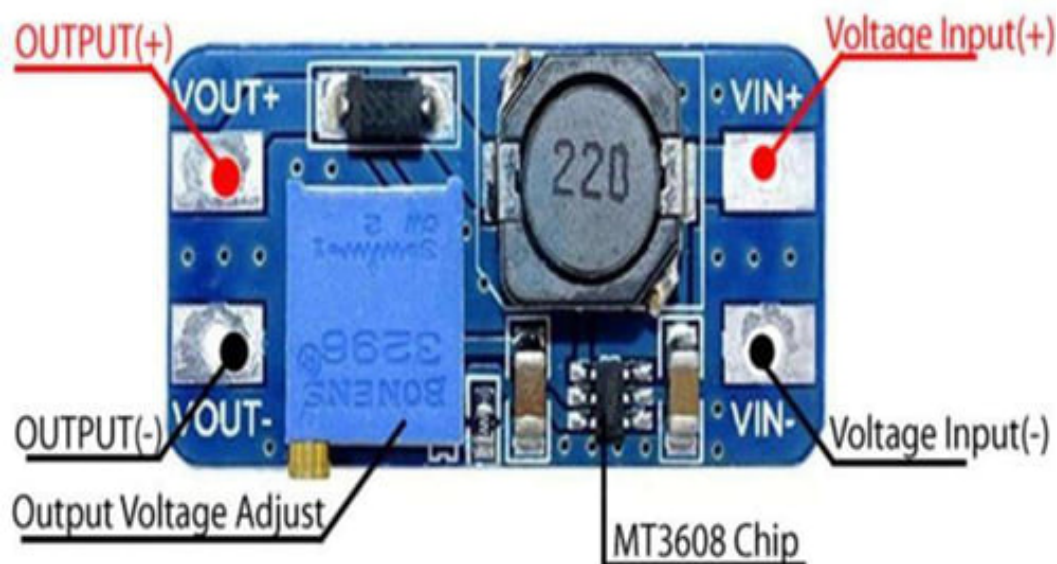


Figure 3.6: MT3608 voltage booster module

- LM2596 Buck Converter: The LM2596 buck converter lowers the power rail voltage from 12V to the voltage required by the ESP-01 and relay module, which is either 5V or 3.3V. The output voltage remains constant and protected from over-voltage conditions.

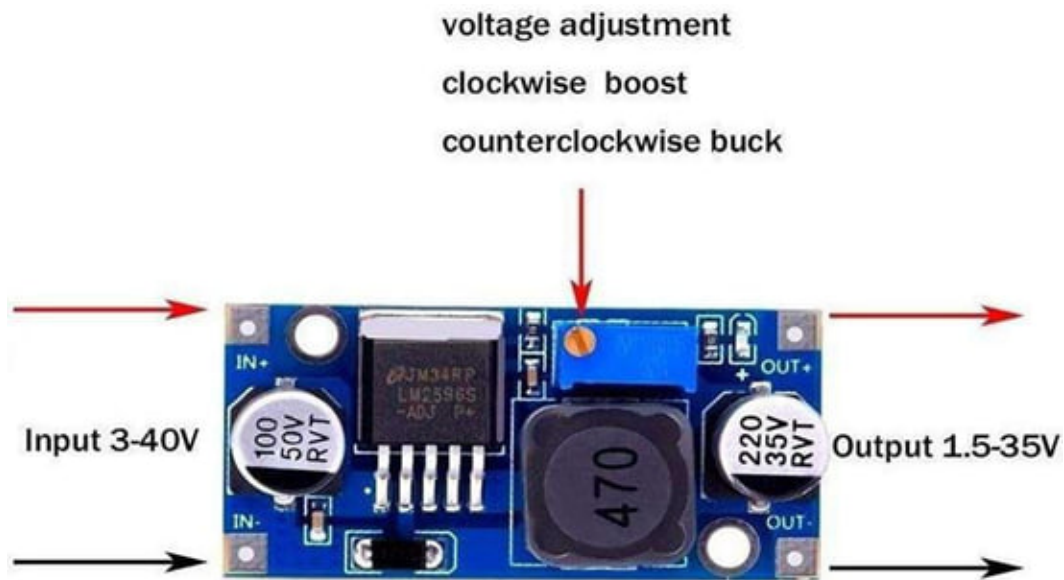


Figure 3.7: LM2596 voltage step-down module

3.2.4 Additional Components

- LED Indicators: LED lights are employed to indicate whether or not our components have power.
- Switches & Jumpers: for the manual operation of the device in the event of any malfunction.

3.3 Software Requirements

3.3.1 Development Tools

- Arduino IDE : Arduino Integrated Development Environment is employed in writing the codes for the ESP-01. The software offers support for ESP8266 boards via the Board Manager along with enabling the inclusion of different libraries such as Firebase and WiFiManager. Besides, there is also an option of having the Serial monitor.
- Android Studio : Android studio was used for developing a mobile application capable of interfacing with the Firebase server. The app provides users (farmers or system administrators) access to controlling the spraying system remotely along with getting updates on its operational status.

3.3.2 Communication Libraries

Firestore Client (Mobizt): This library helps in establishing two-way asynchronous communications between the ESP-01 and the Firebase Realtime Database server. Furthermore, Firestore client also offers security features and ensures reliable data synchronization through the use of JSON structure.

WiFiManager (by tzapu): This library helps to dynamically configure the Wi-Fi connection through the use of a captive portal interface where the user enters the network information which is later stored on the EEPROM.

ESP8266WiFi : Core library used for handling wireless communication capabilities.

3.3.3 Data Handling

- ArduinoJson: a fast and efficient library used to parse JSON files. It helps to handle data received from and sent to the cloud (Firestore). The main advantage of this library is that it works quickly and uses minimal memory space.

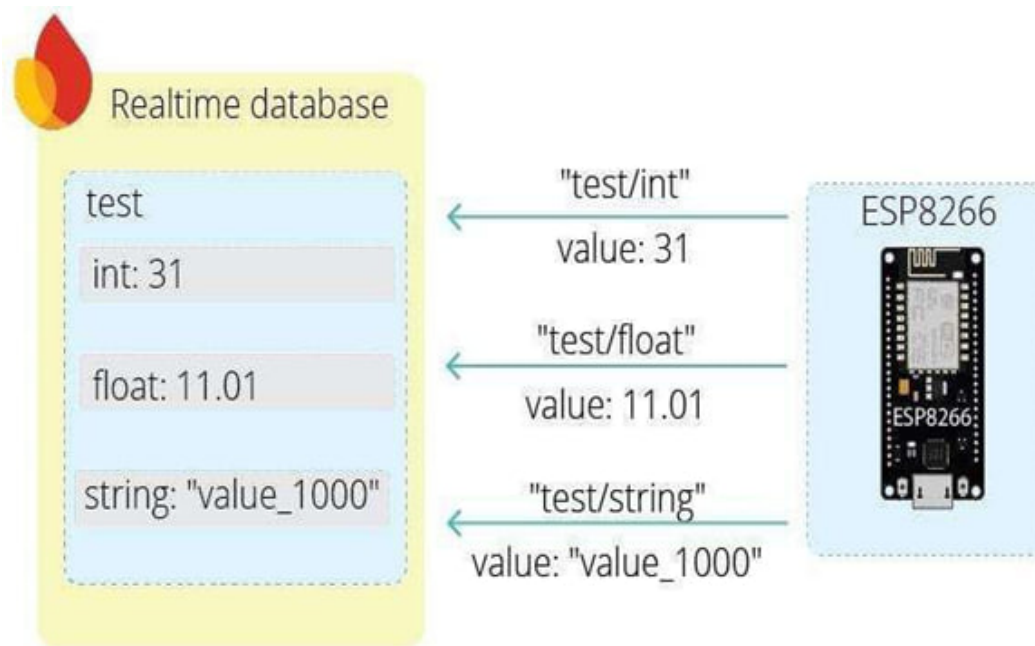


Figure 3.8: ESP8266 – Firestore RTDB architecture

3.3.4 Backend & Mobile Interface

1.Firebase Console: Firestore offers serverless backend capabilities, including Real-time Database, authentication and hosting services. All sensor data and command values are stored using JSON tree structure that supports hierarchical control logic. Firestore Authentication is utilized for access

management and Firebase Security Rules are defined for limiting users' reading/writing permission to nodes.

2. Custom Mobile App (Android): a custom-designed mobile application based on Android Studio includes an intuitive graphical user interface where users can control various spraying functions by pressing buttons. Command values are transmitted to specific Firebase nodes and subsequently received by ESP-01 module from these nodes. Further updates may include logging of data, status information about the device, and authentication capability for users.

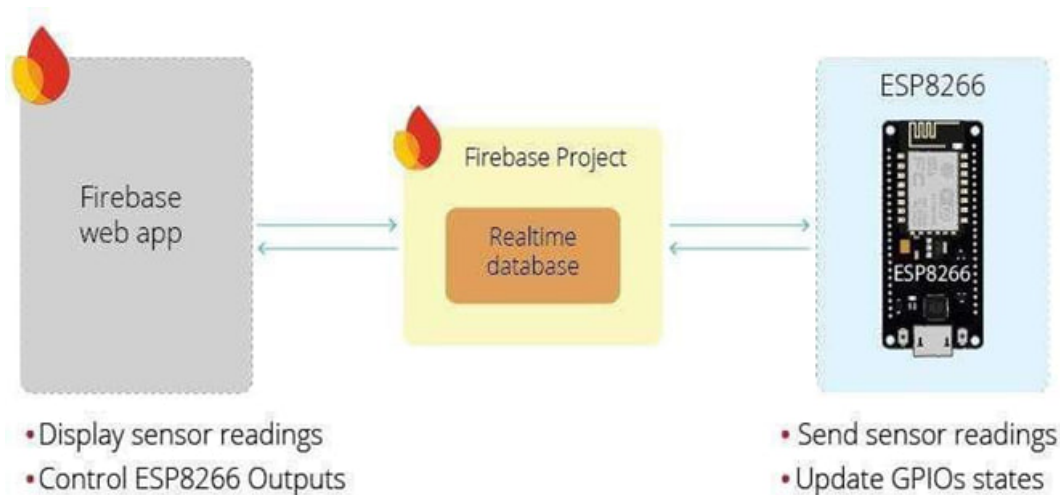


Figure 3.9: System backend architecture

3.3.5 Debugging Tools

1. Serial Monitor: The Arduino IDE is composed of the Serial Monitor which monitors logs being generated by the ESP-01 in real time.

2. ESP Exception Decoder: The ESP-01 tends to crash at times and provides an undecipherable stack trace. This decoding tool converts the gibberish to something meaningful by interpreting function calls and their corresponding files.

3. Android Studio Logcat: Logcat shows all logs, exceptions, and other system information during the execution of an android application. Issues such as connection errors with Firebase and lag in the user interface become obvious when using this tool.

3.4 Functional Architecture

The intelligent spray system consists of four separate layers where each layer is responsible for performing a particular function within the data pipeline. This modular architecture ensures scalability and focuses on performing a particular task by each layer.

3.4.1 Application Layer (User Interface)

The user interacts with the application. The application has a very basic user interface which allows the user to switch on/off the pump and solenoids, and any activity done writes an appropriate number such as "1" or "0" into that particular field in the Firebase database.

The entire IoT framework is hidden behind this simple application layer which makes sure that everything is intuitive for the user regardless of his/her technical expertise.

3.4.2 Cloud layer (Firebase)

The cloud layer holds the real-time data, which keeps all the devices synchronized through Firebase in real-time. The user makes a request through the application, Firebase updates the particular field, and it is read by the ESP-01 module in near real-time. This mechanism results in the near real-time speed.

In addition, the cloud layer provides opportunities for collecting historical data and conducting analytics.

3.4.3 Control layer (ESP-01)

ESP-01 stays online and gets information about the particular field through the callback mechanism. When new values appear, they are read, and the GPIO pin is either set as HIGH or LOW depending on that. The switch turns the relay and all subsequent devices on and off.

Connection issues are covered by the safety mechanism of reconnecting automatically to the network.

3.4.4 Actuation layer (Relays, pumps, solenoid valves)

This layer represents actuating devices, such as relays that isolate the power and control lines, and actual pumps or valves that perform the watering operations.

For future developments, it is possible to add LED indicators that will ensure correct operation of the actuation process and monitor the system state.

CHAPTER 4

Model Proposed

4.1 Introduction

IoT-enabled irrigation and spraying systems have not been entirely unheard of before; there is an ample amount of literature regarding these, and most of it relies on the ESP8266 microcontroller platform. Before embarking on our journey, we have perused through various guides and literature related to these projects, primarily for understanding what methods have been tried previously and where did they fail.

Our most helpful source was an instructive video made by RandomNerdTutorials on IoT communication control of ESP8266 GPIOs via Firebase. This provided us with the basic foundation regarding cloud communication, which saved us a lot of time and effort in dealing with the libraries involved. Apart from this, almost all other resources we came across used the bulky NodeMCU boards or costlier platforms, thus justifying the ESP-01 selection for this project.

4.2 System Overview

The Smart Spraying System is an IoT-based system, which comprises of four main subsystems.

1. Control Unit:

In the ESP-01 lies at the heart of the entire system. The ESP-01 is a small-sized module based on the ESP8266 chip, whose task is to monitor all incoming signals from the cloud and control the appropriate hardware in accordance with them. Even despite its small size and limited GPIO pins, it successfully copes with the work that needs to be done in this system. Its basic tasks include:

- Establishing Wi-Fi connection using the wifi libraries included in ESP
- Reading/writing data in the Firebase Realtime database using the Firebase ESP Client library
- Executing conditional logic in order to control the relays of the pressure pump and the solenoid valve

2. Spraying Mechanism:

This subsystem constitutes the mechanical actuator in the system. These include:

- 12 V DC pressure pump that creates the force necessary to push the fluid through a nozzle directed to a particular plant or crop.
- 12 V DC solenoid valve that regulates the flow by opening and closing its flow line.
- One pipe inlet that is used as a conduit from an external reservoir that contains either water or any other sprayable fluid.

The important aspect about the spray mechanism that should be mentioned is that there are no separate tanks for different fluids. Instead, the entire system uses one unified inlet pipe. The operator simply fills an external tank with whatever fluid is required and then connects it to the input pipe.

3. Communication interface:

The role of the communication interface subsystem is to maintain the connection between the application, Firebase, and the ESP-01 as stable as possible:

- the WiFiManager library makes it possible for the ESP-01 module to be assigned the Wi-Fi password without being hardcoded into the ESP-01
- the Firebase ESP Client allows asynchronous updates from the cloud
- the entire architecture is designed to minimize latency and make the commands nearly instantaneous.

4. User Interface:

The mobile application forms the front end used by the user to control everything. This application, made in Android Studio, interacts with Firebase to issue orders and receive status reports. These include:

- selection of spray mode (like "Start Spray" or "Stop Spray");
- real-time monitoring of operation status; and
- feedback regarding connectivity and execution status of the device.

All these subsystems function as an integrated unit to form the entire platform. This system is practical enough to be implemented in urban gardens, greenhouses, and even small farms where implementing any sort of automation just isn't practical or affordable.

4.3 Block Diagram

The following block diagram represents how each and every part operates in harmony with others:

- Mobile app → writes user commands to → Firebase
- Firebase → provides updates to → ESP-01 (Wi-Fi connection)
- ESP-01 → activates → Relay Module
- Relay module → activates → Solenoid Valve / Pressure Pump
- Inlet (Water/Pesticide/Fertilizer) → passes through → Valve → Nozzle → Target area

4.4 Working Principle

The entire process operates via continuous real-time control, which is based on the cloud. Below is an explanation of what happens at each step.

4.4.1 User Command Initiation

A command issued by the user from within the mobile application entails pressing a button such as “Start Spraying.” This sends a key-value pair entry in the Firebase Realtime Database via JSON.

4.4.2 Firebase Modification

This command triggers modification of a certain key entry in the Firebase database in real-time (e.g., `spray_status`: “on”).

4.4.3 Data Polling by the ESP-01

The ESP-01 device always listens to any changes made to the database via the Firebase ESP client library; therefore, there is no need for data polling.

4.4.4 Logical Decision Making

Regardless of the value held by `spray_status`, the microcontroller takes this value into consideration, determining whether the GPIO connected pin should be set to HIGH or LOW.

4.4.5 Activation of Relay

Relay activates; it energizes both pressure pump and solenoid valve simultaneously. As a result, liquid is being taken inside and sprayed out from the nozzle through one action.

Spraying Procedure

Continuous spraying is done unless the user issues a "stop" command through the application or until the system times out (automatic system shut-down).

4.4.6 System Shutdown

GPIO pin is set to LOW, the relay goes off, spraying is turned off. It's as simple as that. The overall procedure allows you to have total remote control of your device, making usage highly efficient and reducing wastage significantly.

4.5 Control Logic and Flowchart

ESP-01 executes its control logic according to the conditions that it senses from Firebase. All code has been programmed in the Arduino IDE, making use of non-blocking code for real-time operation. Pseudocode of the control logic has been shown below.

```
Start:
→ Initialize Wi-Fi using WiFiManager
→ Connect to Firebase
→ Loop:
    → Read 'spray_status' from Firebase
    → If 'spray_status' == "on":
        → Set GPIO HIGH
        → Start Spray
    → Else if 'spray_status' == "off":
        → Set GPIO LOW
        → Stop Spray
    → Delay(500ms) and repeat
```

Figure 4.1: Control logic (pseudocode)

4.6 Benefits of Proposed Model

4.6.1 Low-cost Hardware Design

All components utilized here have been selected based on cost considerations. ESP-01 board costs a fraction of that of conventional PLCs or industrial controllers, and all other equipment, such as Firebase, Arduino IDE, Android Studio, are entirely free.

4.6.2 Reduced Plumbing Complexity

The inlet piping arrangement replaces what would otherwise consist of several tanks and valves, making the system less complex and cheaper. Real-Time Cloud Control Using Firebase offers real-time synchronization of the control application and ESP-01, allowing commands to be acted upon almost immediately.

4.6.3 Wireless Operation

Provided that the ESP-01 has access to the Internet, the device could be controlled remotely, making it a good choice for farm and greenhouse installations that may not have someone onsite regularly.

4.6.4 Scalability

The flexibility of the hardware means that it can later integrate sensors such as:

- Soil moisture sensors – for condition-based application;
- DHT11/DHT22 sensors – adding the ability to monitor temperature and humidity; and
- Rain sensors – preventing spraying in case of precipitation.

4.6.5 Power Management

Power is consumed when the system is actively spraying. In idle mode, the ESP-01 barely uses any power at all, and the relay and pump remain switched off.

4.6.6 Intuitive User Interface

The application is easy to use even for non-expert users.

All features related to connectivity, controls, and real-time information can be found in one screen.

4.6.7 Safe Hardware Usage

The relay ensures that the high power part, which consists of the pump and valve, is fully separated from the low-powered ESP-01. This separation makes sure there is no risk of damage and extends the lifespan of the entire system.

The system uses the following utilities:

- The Serial Monitor;
- The ESP Exception Decoder; and
- Logcat in Android Studio.

CHAPTER 5

Experimentation and Results

The design process was quite challenging due to the numerous tests that had to be carried out in order to find appropriate components that would work effectively, reliably, yet within budget. Various hardware setups had to be tried out, along with software configurations and various architectures.

This entire procedure is discussed in this chapter. It discusses microcontroller selection, power supply configuration, relay and valve performance tests, as well as durability tests of the system under harsh conditions. Throughout, the goal was to develop a quick response, energy efficient, and easily scalable system.

5.1 Microcontroller Assessment

The choice of microcontroller was possibly the most important decision in the construction process. The board had to be able to communicate with Firebase, understand instructions, and initiate actions through the actuator. Many boards underwent testing before one emerged victorious.

5.1.1 ESP-32

The ESP-32 became our first candidate, mainly due to its popularity among IoT devices. It showed high compatibility with Firebase and came with a dual-core processor; however, there were some factors that made using it complicated:

- unreliable connections during flash and programming processes;
- power wastage through unused features, including Bluetooth;
- limited GPIO pin availability in certain boards; and
- increased power consumption.

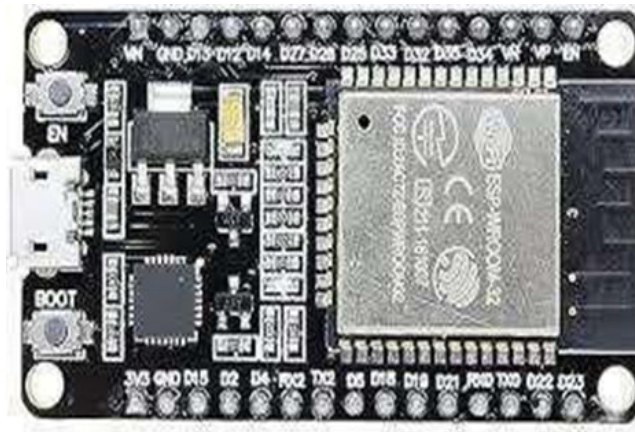


Figure 5.1: ESP-32 development board

5.1.2 AiThinker ESP32-CAM

This module came with inbuilt camera functionality and more GPIO capabilities. Nonetheless,

- the camera used considerable amounts of memory;
- Firebase actions took time to respond; and
- it was costly and complex, especially since we did not need any vision-related application.



Figure 5.2: AiThinker ESP32-CAM module

5.1.3 Raspberry Pi 3B+

It provided great flexibility with its own OS (Linux) as well as numerous expansion capabilities.

But very soon it was decided to give it up due to the following reasons:

- its slow boot time was a serious drawback, making it less suitable for actuation purposes;
- it used SD cards, which could be easily damaged;
- it consumed more energy; and
- it was simply excessive for our project.



Figure 5.3: Raspberry Pi 3B+

5.1.4 NodeMCU

NodeMCU was easy to code and had plenty of GPIOs available. But it was:

- relatively bigger in size than the ESP-01;
- more power-consuming than the ESP-01; and
- good in terms of performance, but not better than ESP-01 while using Firebase with little intensity.

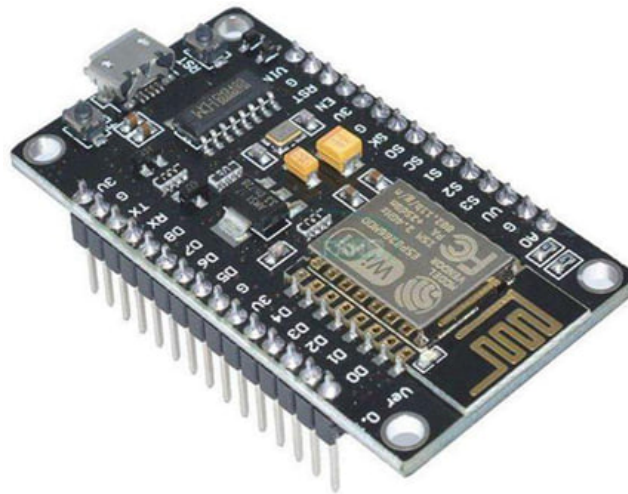


Figure 5.4: NodeMCU development board

5.1.5 H5 Stack

The development kit came with an excellent interface and a screen, but it had some disadvantages as well, namely:

- it was expensive for such a basic spraying device;
- the screen and other accessories were unnecessary for our needs; and
- it consumed a lot of power.

5.1.6 Final Choice: ESP-01

The following criteria led to the choice of the ESP-01:

- compact and cost-efficient;
- stable performance with Firebase in conjunction with lightweight libraries

(Firebase ESP Client, ArduinoJson);

- GPIO pins available (just a single relay output needed);
- stability of the Wi-Fi connection and its quick recovery after failures; and
- efficient operation with optimized code and minimal hardware components.

5.2 Power Supply Testing

Power supply was vital for the successful operation of this project. Given that various devices (such as relay, ESP-01, solenoid valve, pressure pump) had different voltage levels. multiple combinations were evaluated.

5.2.1 AC to DC Adapters

Various AC to DC adapters were tried out:

- 5V/1A DC Adapter: Could not support running both the ESP as well as the relays simultaneously. Relays would not switch on with load.
- 5V/2A/3A DC Adapter: Was better than previous one, but not adequate for running the solenoid and pump simultaneously.
- 12V DC Adapter: Worked reasonably well for running relays and small loads.
- 24V DC Adapter: Sufficient power for running heavy duty solenoids, however risky for use with ESP-01.
- 220V AC: Not used

5.2.2 Voltage Regulation

To suit all components:

- Boost Regulator (MT3608) to boost voltage from 12V to 24V for solenoid
- Buck Regulator (LM2596) to step-down voltage from 12V to 3.3V for ESP-01

Such an approach was effective in regulating the voltage.

5.2.3 Final Power Configuration

Our final power configuration consisted of:

- 12V 2A DC power adapter (AC to DC) as the main power supply;
- LM2596 buck converter providing power at 3.3V to the ESP-01;
- MT3608 boost converter supplying power to the 24V solenoid valve;

- direct 12V power to the DC pressure pump.

We did not implement battery power due to voltage fluctuation problems that Li-ion batteries present, which may damage the module.

5.3 Relay Trials

5.3.1 Relay Module Testing

Tests were conducted on multiple relay modules:

- general-purpose single-channel 5 V relay;
- opto-isolated relays; and
- specific ESP-01 relay module, the best fit proved to be.

Tests for:

- dry contact operation showed millisecond response from the relays;
- load operation demonstrated reliable actuation of solenoids and pumps continuously; and
- current consumption was satisfactory within power supply limits.

Testing Solenoid Valves

Various solenoid valves were tested for efficiency. They included:

- 12V DC solenoid valve: had quick response but had poor flow rate in cases where the water pressure was high;
- 24V DC solenoid valve proved to be the most efficient. It had a tight seal and its opening process was quick and neat;
- 220V AC solenoid valve was slow, noisy and required additional insulation for safety.

Conclusion: The 24V DC solenoid valve was the most effective. It responded quickly and also worked perfectly with the voltage booster in place.

5.3.2 Pressure Pump Tests

Different types of 12V pumps running on DC power were tested and finally selected on account of:

- a constant flow of water ideal for spraying;
- ability to withstand dry operation periods; and
- performance when working with the load generated by the relay module.

5.4 Connectivity and Cloud Communication Test of ESP-01

ESP-01 was tested under several conditions regarding Firebase testing:

- Wi-Fi connection time: four to six seconds from the initialization using WiFiManager;
- Data synchronization time: Firebase commands sent to the ESP-01 within less than one second;
- Stability: stable Wi-Fi connection for more than two hours continuously;
- Security: Firebase token provided secure data transmission; and
- Reconnection: ESP-01 reconnected within fifteen to twenty seconds after a loss of Wi-Fi connectivity.

There were no data losses or delays of any commands during over thirty tests.

5.5 Cycle Tests For The Spraying System

A series of complete cycle tests were performed to ensure reliability in the following aspects:

- Spray system activation through manual intervention on the mobile application linked to the Firebase database.
- Activation of liquid flow immediately after the ESP command through relay action.
- Built-in timeout that automatically turns off the spray after a certain period (e.g. 10 seconds).

5.6 Environmental and Field Adaptability Tests

Field adaptability tests were performed to check how the system coped with various environmental conditions:

- Waterproof container: the system components were enclosed within a waterproof plastic container.
- Dust filtering: a simple filter mechanism added to the nozzle to prevent clogging.
- Temperature limits: reliable operation from 15°C to 40°C.
- Wi-Fi limitations: reliable transmission up to 20 meters; additional signal repeaters can extend this distance

Thus, testing proved that the system could be effectively deployed either in an urban environment or out in the field.

Summary of Findings

Table I: The Results Summary of Findings

Table 5.1: The Results Summary of Findings

Component	Final Choice / Result
Microcontroller	ESP-01 with Firebase ESP Client
Power Source	12 V 2 A DC adapter with buck (LM2596) and boost (MT3608)
Relay	ESP-01 relay module
Valve	24 V DC solenoid valve
Pump	12 V DC pressure pump
Cloud	Firebase Realtime Database, responsive & secure
Connectivity	Stable within 20 m; fast reconnection
Environmental Setup	Waterproof casing, dust mesh, temperature-tolerant

At each test stage, the project moved away from an idea to a working model of the product. Optimization did not just take place for its functionality but for its usability in real-world applications, whether urban or rural agriculture. The result is a performant and scalable solution.

CHAPTER 6

Conclusion

Conclusion

In general, we were happy with the results of our efforts. As it was mentioned at the very beginning, the main task was to construct a practical, rather than a theoretical solution. And we believe that it was fulfilled.

The choice of the board proved correct. We tried several other models prior to using ESP-01 but as was shown by the testing of Chapter 5, the choice was justified. Used in combination with Firebase and an Android app, the system operated successfully in all test cycles. Speaking about the problems, there was some difficulty concerning the selection of optimal parameters for powering the sensors and the board. Some components failed before we could establish a correct set of parameters, however, the final one using a 12V 2A source along with LM2596 and MT3608 did its job well.

Concerning the improvements for future work, we think that one should change the plumbing design of the prototype as having multiple inlets with individual solenoids would significantly increase the practical value of the system.

It is true that the modular approach proved beneficial here. It would have been challenging to switch to another model for the 12V solenoid valve. However, thanks to its modularity, switching to the 24V valve was relatively easy.

6.1 Pros and Cons

Pros

- Cheap parts cost – the total cost for ESP-01 & relay module is less than Rs.200.
- The system can be controlled from anywhere on internet.
- The system was successfully tested even from outside of college premises.
- Easy maintenance.

- All parts are readily available in Lajpat Rai Market.
- Minimal idle power consumption when there is no spraying process.
- Firebase Spark Plan was free.

Cons:

- ESP-01 comes equipped with just two GPIO ports that can be used.
- Incorporating additional sensors was challenging owing to the insufficient number of GPIO ports.
- Entire operation relies entirely on WiFi connectivity.
- There is no alternative plan in place in case the internet connection fails.
- The mobile application does work; however, its user interface is very simple.
- UI enhancement was not taken into account during development.

6.2 Future Scope

The system functions as intended but can be expanded. The following are some potential directions we could go in:

1. Sensor Integration

Some additional features that would be useful if we pursued this project any further:

Currently, one major drawback of the system is the lack of awareness of the environmental situation around it. An incorporation of even a simple soil moisture sensor would make it possible for the ESP-01 to autonomously determine whether spraying is necessary. Since we're limited by the number of GPIO pins, we might want to switch to an ESP-32 for such functionality, which in turn provides an alternative method for controlling it via Bluetooth.

A weather API integration would also be helpful. We had to test a couple of times when we used the spraying machine and it began to rain; that is completely a waste of water and other materials used. An API from OpenWeatherMap could solve this issue.

Integration with solar power will make it usable in fields far away from the electricity grid. The amount of idle power used by ESP-01 is very small, which means a solar panel and battery pack can effectively power it.

As for the application, currently, there is an on/off toggle for turning the machine on and off; however, more features such as scheduling and logging need to be implemented.

6.3 Final Remarks

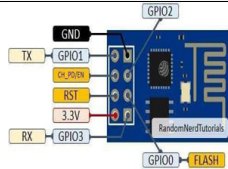
The system demonstrated above proves what can be achieved by utilizing the concepts of embedded systems, IoT, and cloud computing on the basis of a practical application. This is currently a reliable platform which will only get better once some additional fine tuning and inclusion of environmental monitoring devices take place, making this system a complete precision agriculture technology solution.

CHAPTER 7

COMMERCIAL COMPARISON

The following table contrasts a commercially available smart irrigation control product to the custom designed SPRINT prototype and highlights their relative strengths.

Table 7.1: Commercial Comparison of HiOazo Smart Controller and SPRINT

Commercial Alternative (HiOazo Smart Controller)	SPRINT (Proposed System)
	
Cost: Rs.6,202.00	Cost: Highly affordable. Uses budget-friendly components like the ESP-01, which costs a fraction of traditional controllers.
Ecosystem: Closed and proprietary. If a component fails, the entire unit often requires specialized repair or full replacement.	Ecosystem: Modular and open-source. Inexpensive, easily replaceable components (relays, valves, controllers).
Plumbing Complexity: Localized controller; often requires proprietary commercial valves separately.	Plumbing Complexity: Simplified. Single pipe inlet reduces complexity, cost, and maintenance overhead.
Customizability: Locked software with predefined features (4 zones, 16 plans).	Customizability: Highly adaptable to different scales, zones, or conditions. Future sensor integration easily achievable due to modular architecture.
Target Audience: Premium home users with standardized garden setups.	Target Audience: Small-scale farmers or urban gardeners who want entry-level automation without spending a fortune.

APPENDIX

Overview of technologies used.

A.1 Source Code Snippet

ESP-01 Firmware Initialisation:

```
#include <ESP8266WiFi.h>
#include <Firebase_ESP_Client.h>
#include <WiFiManager.h>

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

void setup() {
  Serial.begin(115200);
  WiFiManager wifiManager;
  wifiManager.autoConnect("SmartSprayerAP");

  config.api_key = "<API_KEY>";
  config.database_url = "<DATABASE_URL>";
  auth.user.email = "<EMAIL>";
  auth.user.password = "<PASSWORD>";
  Firebase.begin(&config, &auth);
}
```

Figure 7.1: ESP-01 firmware initialisation

A.2 Mobile App Screenshot

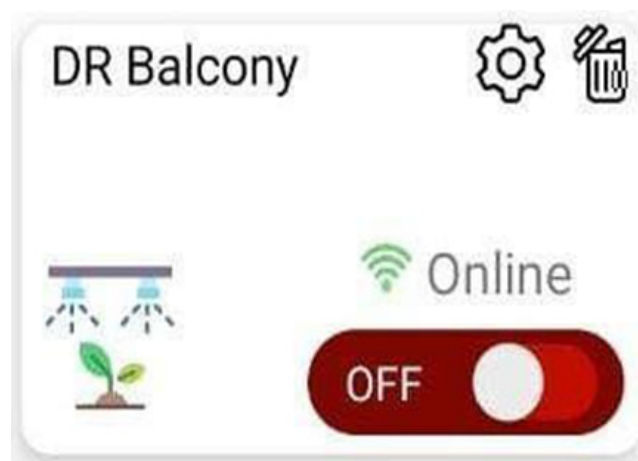


Figure 7.2: Controller card layout

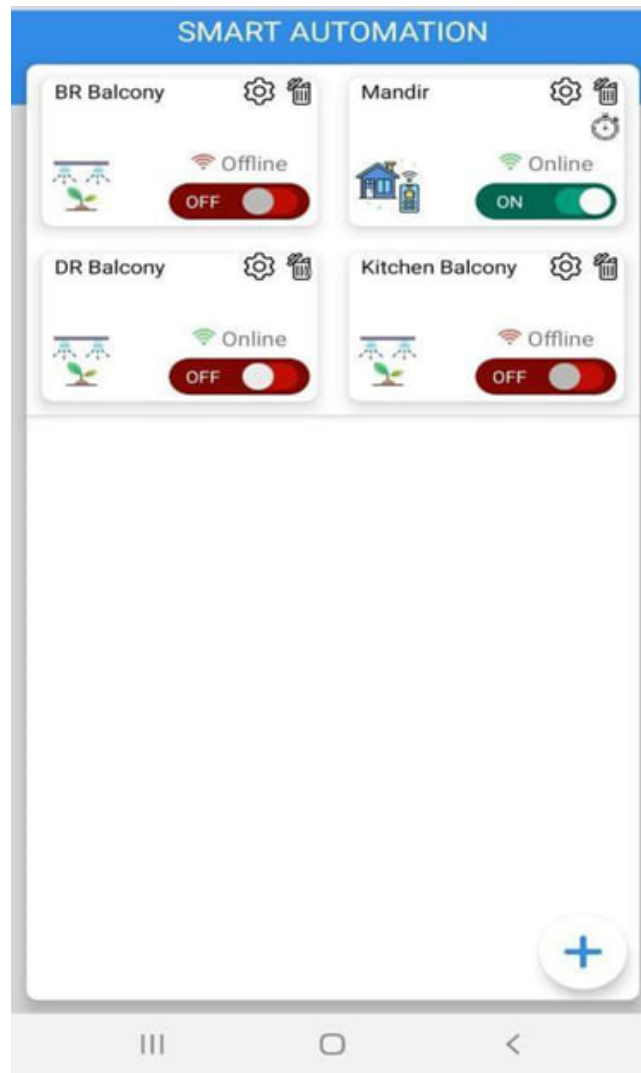


Figure 7.3: Android app user interface

A.3System Circuit Diagram

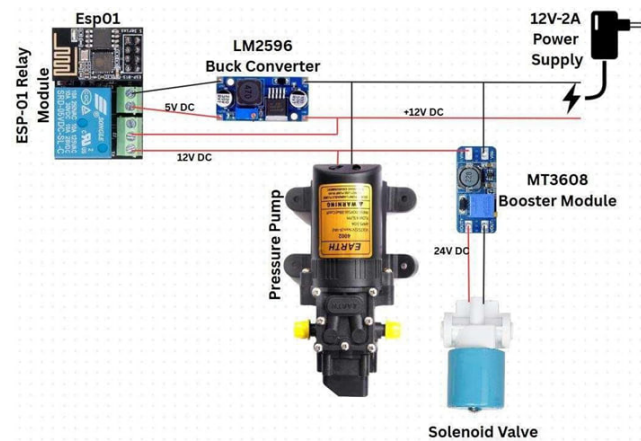


Figure 7.4: Internal circuit diagram

A.4 Components Used

Table II: Components used in the final prototype

Table 7.2: Components used in the final prototype

Component	Specification
Microcontroller	ESP-01 (ESP8266)
Relay Module	Single-channel, ESP-01 compatible
Solenoid Valve	24 V DC
Pressure Pump	12 V DC (mini diaphragm pump)
Power Adapter	12 V 2 A DC
Boost Converter	MT3608 (for 5 V to 12 V / 24 V)
Buck Converter	LM2596 (for 12 V to 5 V / 3.3 V)
Connecting Wires	Male-to-female jumper wires
PCB Board (optional)	For compact circuit assembly
Protective Enclosure	For waterproofing and safety

REFERENCES

- [1] R. Silva and S. Santos, “ESP8266 NodeMCU with Firebase Realtime Database,” Random Nerd Tutorials, Apr. 2023. [Online]. Available: <https://randomnerdtutorials.com/esp8266-nodemcu-firebase-realtime-database/>
- [2] R. Silva and S. Santos, “Firebase: Control ESP8266 NodeMCU GPIOs from Anywhere,” Random Nerd Tutorials, 2023. [Online]. Available: <https://randomnerdtutorials.com/firebase-control-esp8266-nodemcu-gpios/>
- [3] M. Mobizt, “Firebase ESP Client Library for Arduino,” GitHub, 2023. [Online]. Available: <https://github.com/mobizt/Firebase-ESP-Client>
- [4] T. Zapu, “WiFiManager: ESP8266 Wi-Fi Connection Manager Library,” GitHub, 2023. [Online]. Available: <https://github.com/tzapu/WiFiManager>
- [5] Arduino Community, “ESP8266WiFi Library Documentation,” Arduino ESP8266 Documentation, 2023. [Online]. Available: <https://arduino-esp8266.readthedocs.io/en/latest/esp8266wifi/readme.html>
- [6] Google Firebase Team, “Firebase Realtime Database,” Firebase Documentation, Apr. 2023. [Online]. Available: <https://firebase.google.com/docs/database>
- [7] AMS, “AMS1117 Voltage Regulator Datasheet,” 2023. [Online]. Available from electronics component retailers.
- [8] ESP8266 Community, “ESP8266 Forum and Documentation,” ESP8266.com, 2023. [Online]. Available: <https://www.esp8266.com/>
- [9] Manufacturer Datasheet, “Buck Converter Module Datasheet,” 2023. [Online]. Available from the product page.
- [10] Arduino Libraries, “Firebase ESP8266 Client,” Arduino Library Manager, 2023. [Online]. Available: <https://www.arduinolibraries.info/libraries/firebase-esp8266-client>
- [11] A. Nair, “ESP8266 NodeMCU with Google Firebase – Creating a Web App,” Microdigisoft, 2023. [Online]. Available: <https://microdigisoft.com/esp8266-nodemcu-with-firebase-creating-a-web-app/>

- [12] P. Narada, “Connecting ESP8266 to Firebase to Send & Receive Data,” Hackster.io, 2023. [Online]. Available: <https://www.hackster.io/pulasthi-Narada/connecting-esp8266-to-firebase-to-send-receive-data-4adf66>
- [13] Google Firebase Team, “Firebase Authentication,” Firebase Documentation, Apr. 2023. [Online]. Available: <https://firebase.google.com/docs/auth>
- [14] MIT App Inventor Team, “Create Mobile Apps,” App Inventor, Apr. 2023. [Online]. Available: <https://appinventor.mit.edu/>
- [15] S. Gupta, “Firebase Realtime Database with Android Studio – Complete CRUD App Tutorial,” YouTube, Aug. 2022. [Online]. Available: https://www.youtube.com/watch?v=2H7_6Cz29V8
- [16] K. Choudhary, “Android Studio with Firebase – Realtime Data Sync Example,” Medium, Oct. 2023. [Online]. Available: <https://medium.com/firebase-developers/android-studio-with-firebase-realtime-example-453d1b8f6a7c>
- [17] Android Developers, “Build a Simple UI with ConstraintLayout,” Android Developer Guide, 2023. [Online]. Available: <https://developer.android.com/guide/topics/ui/layout/declaring-layout>
- [18] Android Developers, “Add Firebase to Your Android Project,” Firebase Docs, 2023. [Online]. Available: <https://firebase.google.com/docs/android/setup>